Nom et prénom :
Zied Ellili

Année universitaire :
2025 – 2026



École Nationale d'Ingénieurs de Sfax – ENIS

Rapport de Stage Ouvrier

Département Génie Informatique

Sujet :

MovieHub – Conception d'une plateforme de films et séries avec un moteur de recommandation par intelligence artificielle

Encadrant industriel :
M. Firas Ben Hlima

Dates du stage :
13-07-2026 / 09-08-2026

STE KUSA



Table des matières

Remerciements **6**

1 Introduction **7**

2 Présentation de l'Entreprise d'Accueil **8**
2.1 Ste KUSA 8
2.2 Diwan Sport : la plateforme phare 8
2.3 Organisation du département technique 8

3 Présentation du Projet **9**
3.1 Contexte et Problématique 9
3.2 Objectifs du Projet 9
3.3 Public Cible 9

4 Technologies Utilisées **10**
4.1 Vue d'ensemble 10
4.2 Frontend 10
4.3 Backend 10
4.4 Base de Données et API Externes 11

5 Architecture du Système **12**
5.1 Architecture Trois Couches 12
5.2 Flux de Communication 12
5.3 Exemple : Récupération des Recommandations 12

6 Développement du Backend **13**
6.1 Rappel sur les bases – Semaine 1 (du 13 au 15 juillet) 13
6.2 Organisation du Projet Django 13
6.3 Base de Données : MongoDB et MongoDB Atlas 16
6.4 Modèles de Données 17
6.5 Fonctionnalités Développées Côté Backend 17

7 Développement du Frontend **19**
7.1 Technologies du Frontend . . . 19
7.2 Structure du Projet Frontend . . . 19
7.3 Communication avec le Backend et Authentification JWT . . . 20
7.4 Fonctionnalités Développées Côté Frontend . . . 21

8 Système de Recommandation par Intelligence Artificielle **23**
8.1 Approche Générale . . . 23
8.2 Étape 1 : Indexation des Films . . . 23
8.3 Étape 2 : Construction du Profil Utilisateur . . . 23
8.4 Étape 3 : Génération des Recommandations . . . 24
8.5 Résultats de l'Approche IA . . . 24

9 Modélisation de la Base de Données **25**
9.1 Collections MongoDB . . . 25
9.2 Sections Personnalisées . . . 25
9.3 Packs Personnalisés . . . 27

10 Fonctionnalités Principales **29**
10.1 Interface Utilisateur Style Netflix . . . 29
10.2 Intégration des Notes IMDb . . . 30
10.3 Suivi de Progression . . . 31
10.4 Recherche et Découverte . . . 32
10.5 Panneau d'Administration . . . 32

11 Endpoints de l'API REST **35**
11.1 Authentification . . . 35
11.2 Gestion des Films . . . 35
11.3 Recommandations IA . . . 35

12 Défis Rencontrés et Solutions **36**
12.1 Défi 1 : Précision de l'IA . . . 36
12.2 Défi 2 : Sections Incorrectes . . . 36
12.3 Défi 3 : Données Manquantes . . . 36
12.4 Défi 4 : Performance . . . 36

13 Résultats et Statistiques du Projet **37**
13.1 Métriques de Code . . . 37
13.2 Métriques de Performance . . . 37
13.3 Métriques de Contenu . . . 37

14 Compétences Acquises **38**
14.1 Développement Frontend (Angular) . . . 38
14.2 Développement Backend (Django) . . . 38
14.3 Intelligence Artificielle . . . 38
14.4 Bases de Données NoSQL . . . 38
14.5 Outils et Méthodologies . . . 39

15 Perspectives et Améliorations Futures **40**
15.1 Fonctionnalités Prévues 40
15.2 Améliorations Techniques 40

16 Conclusion **41**

Table des figures

5.1 Architecture trois couches de MovieHub 12

6.1 Structure générale du projet (VS Code) 14
6.2 Structure du dossier backend Django 14
6.3 Détail du dossier accounts/ (backend Django) 15
6.4 Détail des dossiers movies/ et recommendations/ (backend Django) 16
6.5 Collections de la base MongoDB visualisées dans MongoDB Compass . . . 17
6.6 Lancement du serveur backend Django (python manage.py runserver) . . . 18

7.1 Structure du dossier frontend Angular 20
7.2 Pages de connexion et d'inscription 21
7.3 Parcours de réinitialisation du mot de passe 21
7.4 Lancement du serveur frontend Angular (npm start) 22

8.1 Section « Recommandé pour vous » générée par le moteur IA 24

9.1 Sections « Films les plus regardés » et « Séries les plus regardées » 26
9.2 Sections « Top 20 Favoris » et « Top 10 Films » 27
9.3 Filtrage des packs par genre 28
9.4 Pack thématique organisé par niveau 28

10.1 Interface principale du tableau de bord MovieHub 29
10.2 Catalogue de films avec pagination 30
10.3 Section « Tendances & Populaires » 30
10.4 Détails d'un film avec affichage de la note IMDb 31
10.5 Section « Films similaires » sur la page de détail 31
10.6 Suivi de progression et section « Continue Watching » 32
10.7 Fonctionnalité de recherche et de découverte 32
10.8 Panneau d'administration de MovieHub 33
10.9 Panneau d'administration de MovieHub 33

10.10Panneau d'administration de MovieHub 34
10.11Panneau d'administration de MovieHub 34

11.1 Extrait du code du moteur de recommandation (Django) 35

Remerciements

Je tiens tout d'abord à remercier chaleureusement **Ste KUSA** de m'avoir accueilli au sein de son équipe pour ce stage ouvrier, et de m'avoir permis de découvrir de l'intérieur le développement de leur plateforme.

Je souhaite exprimer ma profonde gratitude à **Monsieur Alaeddine Derbel**, CEO de Ste KUSA, pour son accueil chaleureux et la confiance qu'il m'a accordée durant toute la période de ce stage.

Mes remerciements les plus sincères s'adressent également à mon encadrant industriel, **Monsieur Firas Ben Hlima**, pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de mon stage. Son expertise technique et sa patience ont grandement contribué à la réussite de cette expérience.

Enfin, je remercie l'ensemble de l'équipe de Ste KUSA pour leur accueil, leur disponibilité et l'ambiance de travail conviviale qu'ils ont su créer durant cette période.

1. Introduction

Dans le cadre de ma formation en Génie Informatique à l'École Nationale d'Ingénieurs de Sfax (ENIS), j'ai effectué un stage ouvrier du 13 juillet au 9 août 2026 au sein de la société **Ste KUSA**, à Sfax.

Ce stage a été pour moi une première immersion concrète dans le monde de l'entreprise et du développement logiciel professionnel. Il m'a permis de mettre en pratique et de consolider mes compétences en développement web full-stack, en gestion de bases de données et en intelligence artificielle appliquée, tout en découvrant le fonctionnement interne d'une startup tunisienne innovante.

Ce rapport présente dans un premier temps l'entreprise d'accueil et le contexte du stage, puis détaille le travail technique réalisé : l'architecture mise en place, le fonctionnement du moteur de recommandation par intelligence artificielle, les fonctionnalités développées, les défis rencontrés ainsi que les compétences acquises tout au long de cette expérience.

2. Présentation de l'Entreprise d'Accueil

2.1 Ste KUSA

Ste KUSA est une startup tunisienne basée à Sfax, spécialisée dans le développement de solutions technologiques pour le secteur du sport et du divertissement. L'entreprise est dirigée par **Monsieur Alaeddine Derbel**, Président-Directeur Général (PDG), qui pilote la stratégie et le développement de la société.

2.2 Diwan Sport : la plateforme phare

Le produit phare de Ste KUSA est **Diwan Sport**, présentée comme la première plateforme de streaming sportif en Tunisie. Diwan Sport permet de diffuser en direct et à la demande les matchs de football des principales compétitions tunisiennes, notamment la Ligue 1 et la Coupe de Tunisie, accessibles depuis un smartphone, une tablette, un ordinateur ou une smart TV [1].

Lancée en mars 2023, la plateforme a marqué une étape importante dans la digitalisation du sport tunisien en proposant un accès à la carte aux contenus sportifs, via des tickets numériques disponibles en kiosque ou par paiement en ligne. Diwan Sport détient les droits de diffusion digitale de ces compétitions et se positionne comme un acteur de référence du streaming sportif en Afrique et dans le monde arabe.

2.3 Organisation du département technique

Au sein de Ste KUSA, j'ai intégré l'équipe technique encadrée par **Monsieur Firas Ben Hlima**, qui a assuré mon encadrement industriel tout au long du stage. Cette équipe est chargée du développement, de la maintenance et de l'amélioration continue de la plateforme Diwan Sport ainsi que des outils internes associés.

1. Source : Tekiano, « Diwan Sport : la première plateforme de streaming sportive en Tunisie est lancée », mars 2023.

3. Présentation du Projet

3.1 Contexte et Problématique

Dans le cadre de ma mission au sein de Ste KUSA, mon encadrant industriel m'a confié la réalisation d'un projet de développement intitulé **MovieHub**. Ce projet, bien que centré sur le streaming de films et séries, m'a permis d'explorer des concepts directement transposables au domaine du streaming sportif : gestion de catalogues de contenu, recommandation personnalisée par intelligence artificielle et suivi de consommation utilisateur – des problématiques également présentes chez Diwan Sport.

Les plateformes de streaming modernes (Netflix, Disney+, Prime Video, et par extension les plateformes de streaming sportif comme Diwan Sport) proposent des catalogues importants, ce qui pousse souvent les utilisateurs à passer du temps à chercher un contenu à regarder. L'objectif du projet MovieHub est de réduire ce temps de recherche grâce à un système de recommandation intelligent et personnalisé, basé sur l'historique et les préférences de chaque utilisateur.

3.2 Objectifs du Projet

— Développer une plateforme de films et séries avec une interface professionnelle inspirée de Netflix.

— Concevoir un moteur de recommandation basé sur l'intelligence artificielle (*Content-Based Filtering*).

— Intégrer des données réelles (notes IMDb, affiches, métadonnées) via des API externes.

— Mettre en place un suivi de progression de visionnage par utilisateur.

— Fournir un panneau d'administration complet pour la gestion du contenu.

3.3 Public Cible

— **Utilisateurs finaux** : passionnés de films et séries souhaitant une expérience personnalisée.

— **Gestionnaires de contenu** : administrateurs gérant efficacement un large catalogue.

— **Développeurs** : apprentissage du développement full-stack moderne avec IA.

4. Technologies Utilisées

4.1 Vue d'ensemble

Le projet MovieHub repose sur une architecture full-stack moderne combinant un frontend Angular, un backend Django et une base de données MongoDB, enrichie par un moteur d'intelligence artificielle basé sur TensorFlow Hub.

4.2 Frontend

Technologie	**Rôle**
Angular 18.2.13	Framework SPA (Single Page Application) avec TypeScript
TypeScript 5.5.4	Langage type-safe
RxJS 7.8.1	Programmation réactive (Observables)
CSS3 (3500+ lignes)	Design responsive inspiré de Netflix

TABLE 4.1 – Technologies Frontend

4.3 Backend

Technologie	**Rôle**
Django 5.0.7	Framework web Python (API REST)
Django REST Framework	Création d'API RESTful
MongoEngine 0.29.1	ODM pour MongoDB
TensorFlow Hub 0.16.1	Modèles IA pré-entraînés
Universal Sentence Encoder v4	Génération d'embeddings (512 dimensions)
NumPy 1.26.4	Calculs numériques
scikit-learn 1.5.2	Calcul de similarité cosinus

TABLE 4.2 – Technologies Backend

4.4 Base de Données et API Externes

— **MongoDB 7.x** : base de données NoSQL orientée documents (MongoDB Atlas, cloud).

— **OMDb API** : métadonnées de films, affiches et notes IMDb.

— **TMDB API** : source de secours pour les notes et affiches.

5. Architecture du Système

5.1 Architecture Trois Couches

Le système est structuré selon une architecture en trois couches (voir Figure 5.1) :

1. **Couche présentation** (Angular 18) : composants Dashboard, Login/Register, panneau Admin et modales de détails.
2. **Couche application** (Django 5) : modules `accounts/` (authentification), `movies/` (gestion du catalogue) et `recommendations/` (moteur IA).
3. **Couche données** (MongoDB) : huit collections regroupant 2 704 films/séries et 105 utilisateurs.

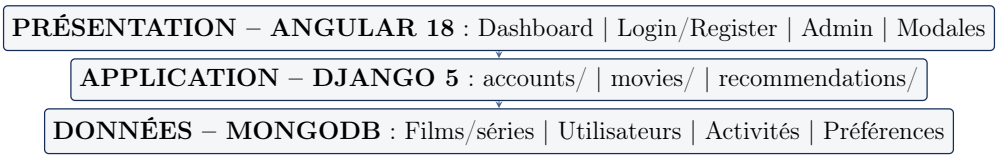


FIGURE 5.1 – Architecture trois couches de MovieHub

5.2 Flux de Communication

```
Action utilisateur -> Composant Angular -> Service HTTP -> Vue Django ->
MongoEngine -> MongoDB -> Reponse JSON -> Angular -> Mise a jour de l'interface
```

5.3 Exemple : Récupération des Recommandations

1. L'utilisateur ouvre le tableau de bord.
2. Le composant appelle `movieService.getRecommendations(username, 100)`.
3. Angular envoie `GET /api/recommendations/{username}/?count=100`.
4. Django reçoit la requête et appelle `RecommendationEngine.get_recommendations()`.
5. Le moteur lit les préférences utilisateur dans MongoDB (favoris, activité, historique).
6. Un embedding utilisateur de 512 dimensions est généré.
7. Une recherche par similarité cosinus est effectuée sur l'index local des embeddings stocké dans `movies.pkl`.
8. Les 100 meilleures recommandations sont retournées en JSON en 200–500 ms.

6. Développement du Backend

Le développement du projet a débuté par la mise en place d'un backend robuste avec Django, avant de passer à la construction de l'interface Angular. Cette partie détaille l'ensemble du travail réalisé côté serveur.

6.1 Rappel sur les bases – Semaine 1 (du 13 au 15 juillet)

— Choix et validation du sujet de stage avec l'encadrant.

— Révision des bases de Python et obtention d'un certificat Coursera « Introduction à Python ».

— Obtention d'un certificat Coursera sur NumPy et apprentissage des DataFrames avec Pandas.

— Discussion du projet avec l'encadrant et démarrage du backend.

6.2 Organisation du Projet Django

Le backend est organisé en plusieurs applications Django indépendantes :

— `accounts/` : gestion des utilisateurs, de l'authentification, des analytics et de la réinitialisation de mot de passe.

— `movies/` : gestion du catalogue de films et séries (création, modification, suppression, recherche, intégration OMDB).

— `recommendations/` : moteur de recommandation par intelligence artificielle et base vectorielle.

Des templates HTML assurent l'affichage des pages côté serveur (connexion, inscription, tableau de bord, catalogue), en complément de l'API REST exposée pour le frontend Angular. Un dossier `user_profiles/` stocke également, en complément de MongoDB, un fichier JSON individuel par utilisateur inscrit, ainsi qu'un dossier `user_reports/` contenant un rapport d'analyse texte par utilisateur généré par le moteur IA.

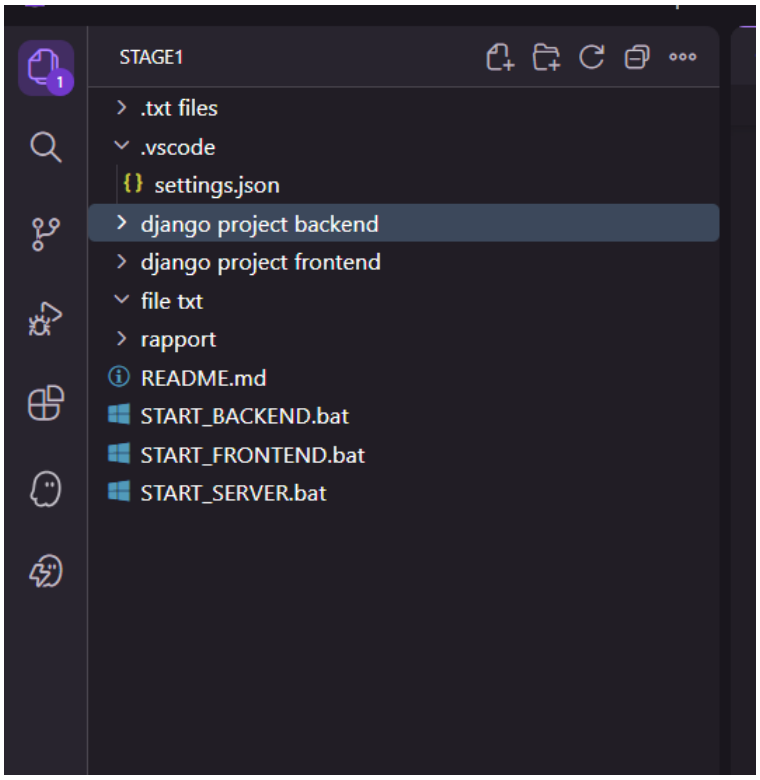


FIGURE 6.1 – Structure générale du projet (VS Code)

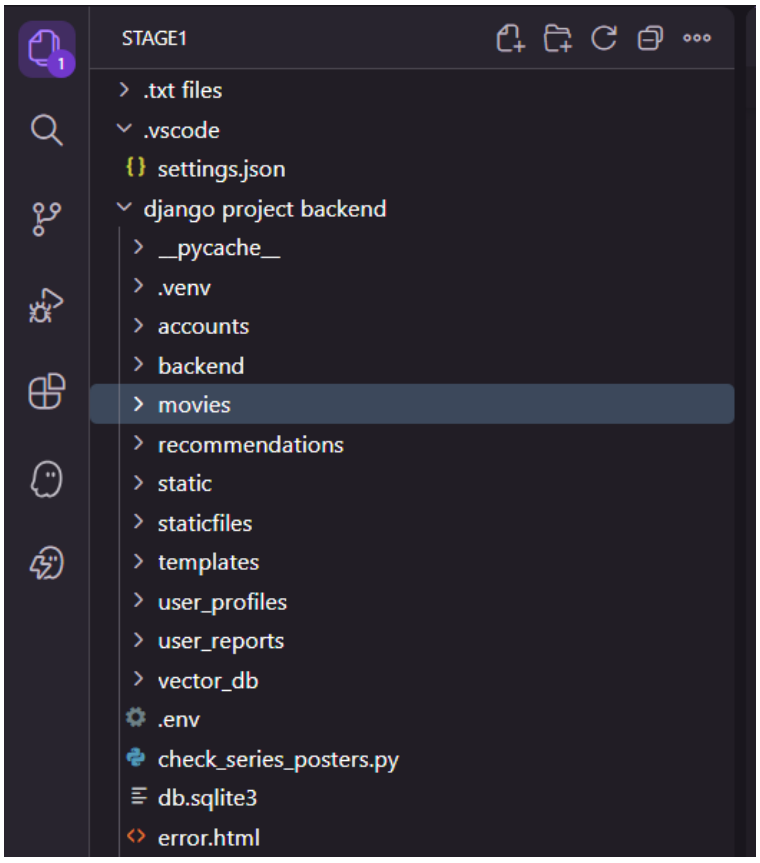


FIGURE 6.2 – Structure du dossier backend Django

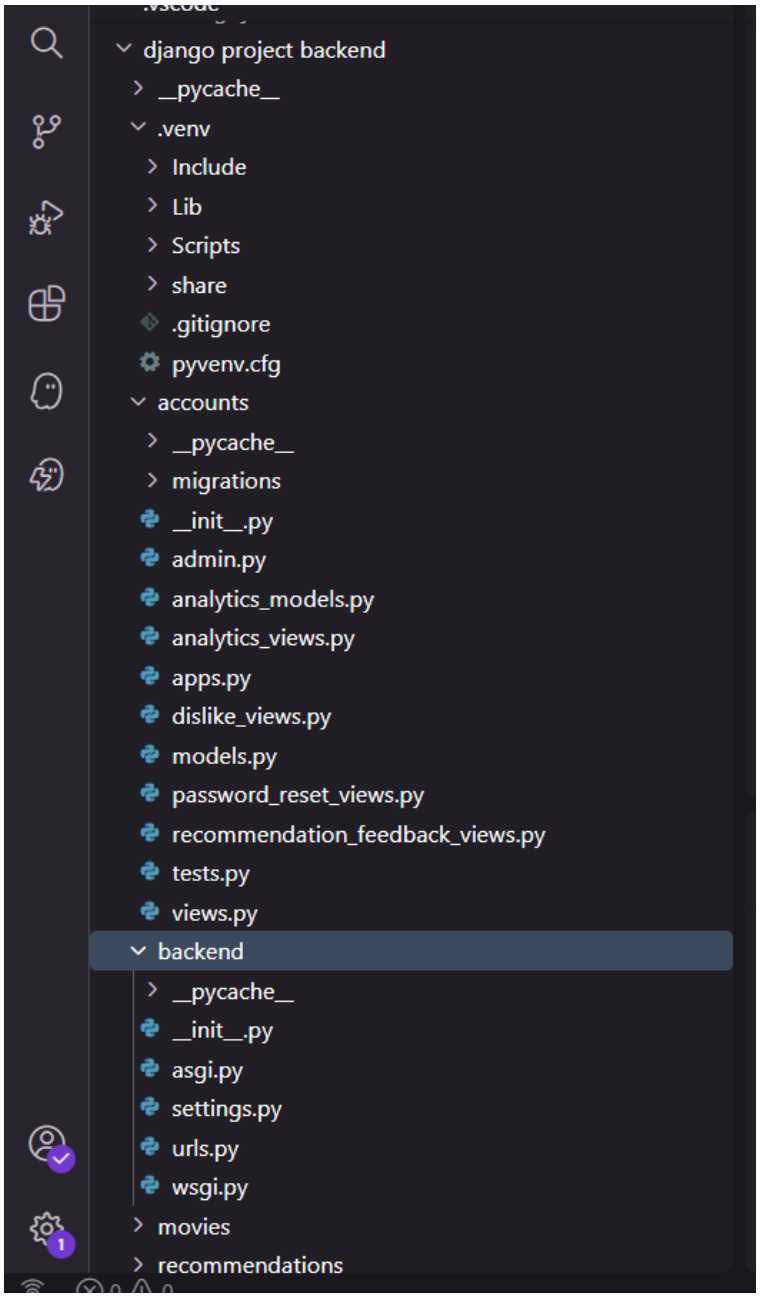


FIGURE 6.3 – Détail du dossier accounts/ (backend Django)

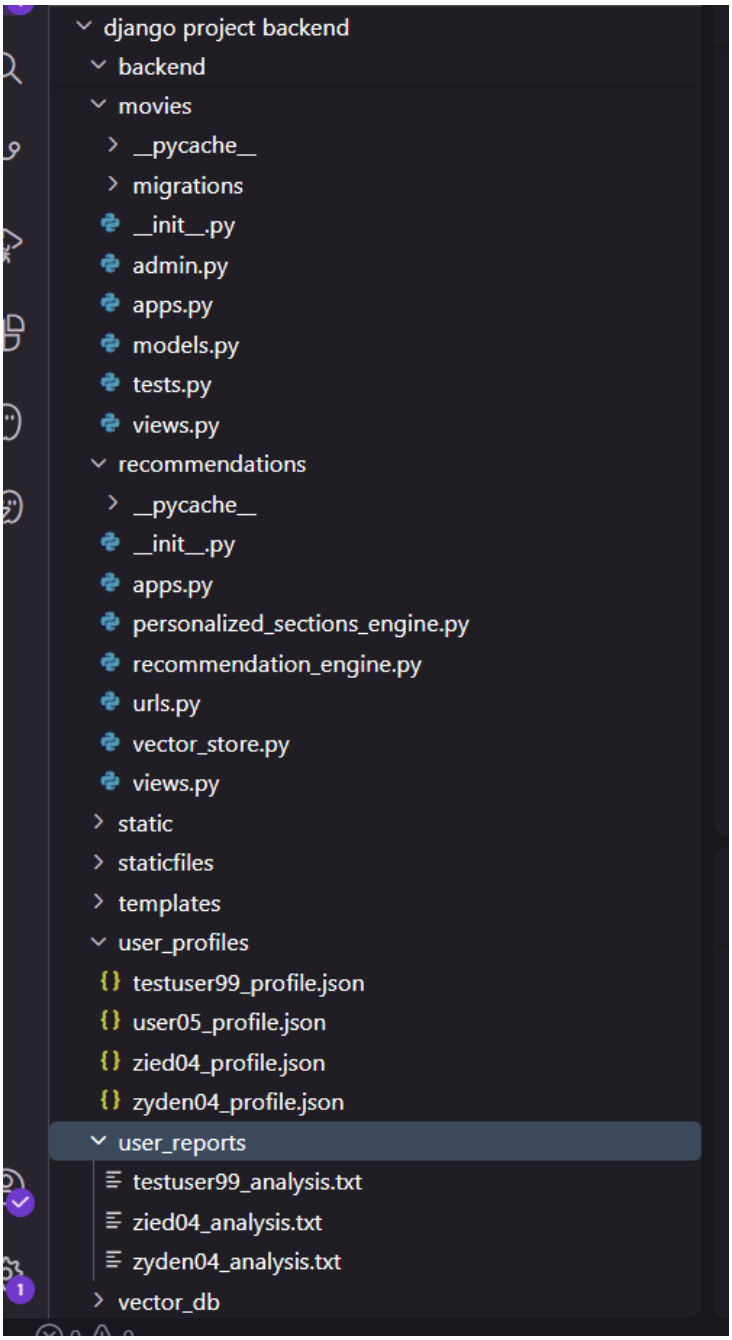


FIGURE 6.4 – Détail des dossiers movies/ et recommendations/ (backend Django)

6.3 Base de Données : MongoDB et MongoDB Atlas

Toutes les données métier (utilisateurs, films) sont stockées dans MongoDB, une base de données NoSQL orientée documents. Ce choix technique se justifie par la nature semi-structurée des données de films : certains champs (par exemple le réalisateur) ne sont pas toujours renseignés, ce qui convient mieux à un modèle NoSQL flexible qu'à un schéma SQL rigide.

La base est hébergée sur MongoDB Atlas (service cloud), via un cluster gratuit créé sur cloud.mongodb.com. La connexion s'effectue dans `settings.py` grâce à `mongoengine.connect()`, en utilisant une URI secrète stockée dans un fichier `.env` (jamais versionné sur GitHub,

afin de protéger les identifiants).

Pour visualiser et interroger les données directement, l'outil MongoDB Compass est utilisé, connecté au même cluster, avec deux collections principales : `users` et `movies`.

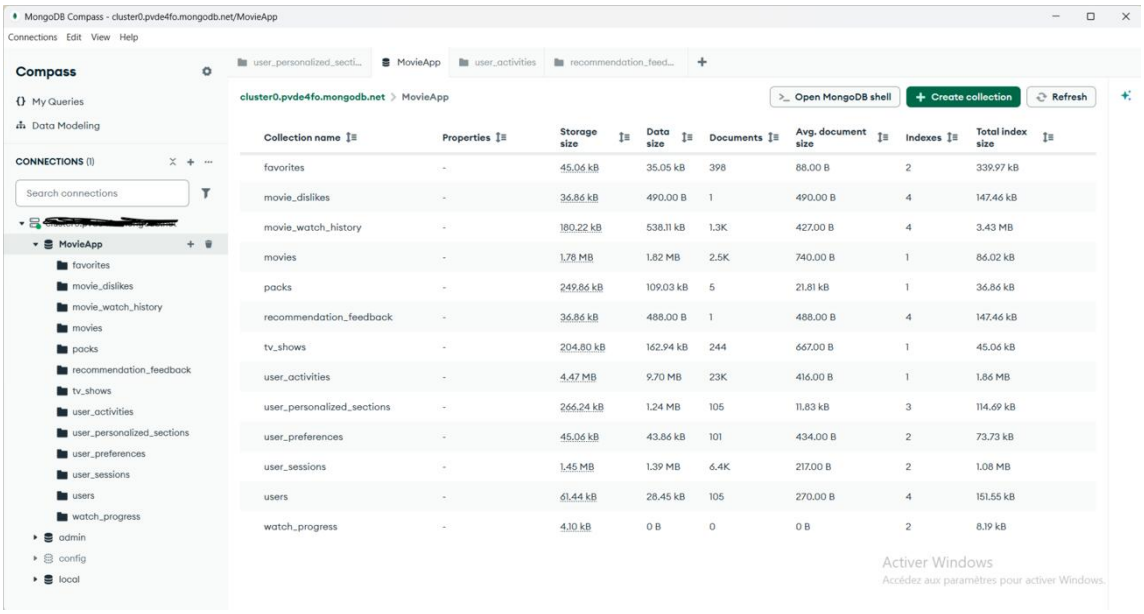


FIGURE 6.5 – Collections de la base MongoDB visualisées dans MongoDB Compass

6.4 Modèles de Données

— **Modèle User** : username, email, user_id (UUID), password (haché, jamais stocké en clair), role (user ou admin).

— **Modèle Movie / TVShow** : show_id, title, type, director, cast, country, release_year, rating, duration, listed_in, description, poster.

— **Modèle UserActivity** : journalise chaque action utilisateur (clics, recherches, vues) avec horodatage, utilisée par le moteur IA pour affiner les recommandations.

— **Modèle MovieWatchHistory** : stocke le pourcentage de visionnage par film et par utilisateur.

— **Modèle UserPreferences** : agrège les genres favoris et le nombre total de films regardés, utilisé comme profil IA.

— **Modèle UserPack** : collections thématiques personnalisées (name, level, genres, movies[]).

6.5 Fonctionnalités Développées Côté Backend

— **Authentification** : connexion et inscription utilisateur/administrateur, mots de passe hachés, gestion des sessions Django.

— **Réinitialisation de mot de passe** : parcours en 3 étapes (email → code de vérification → nouveau mot de passe), géré par `password_reset_views.py`.

— **API REST** sécurisée par JWT (JSON Web Token) pour les clients externes, notamment les endpoints `/api/movies/` et `/api/login/`.

— **Suivi analytique (Analytics)** : chaque action utilisateur (clic, recherche, vue) est journalisée via `analytics_views.py`, permettant de reconstituer l'historique de comportement pour le moteur IA.

— **Packs personnalisés** : création de collections thématiques de films/séries organisées par niveaux et par genres (ex. : pack « Dramahorror », pack « ramdhan2026 »), gérées par le modèle UserPack.

— **Récupération de contenu depuis des API externes** : RapidAPI (Netflix Shows API, filtrée par année de 1950 à 2026) et OMDb (recherche par titre avec détails complets : affiche, acteurs, genre).

— **Tableau de bord administrateur** permettant d'ajouter, modifier et supprimer films, utilisateurs et packs.

— **Sécurité** : protection CSRF sur l'ensemble des formulaires, et isolation des clés API et de l'URI MongoDB dans le fichier `.env`.



FIGURE 6.6 – Lancement du serveur backend Django (python manage.py runserver)

7. Développement du Frontend

Une fois le backend Django stabilisé, le développement s'est poursuivi sur le frontend Angular, qui constitue l'interface visuelle avec laquelle l'utilisateur interagit, pendant que le backend reste la partie invisible qui prépare et fournit les données.

7.1 Technologies du Frontend

— **TypeScript** : langage du frontend, version « typée » de JavaScript qui réduit les erreurs de code grâce à la vérification des types.

— **Angular** : framework structurant l'interface en composants indépendants (fichiers .ts pour la logique, .html pour l'affichage, .css pour le style).

— **HttpClient (Angular)** : module utilisé pour envoyer les requêtes HTTP (GET, POST, PUT, DELETE) vers l'API Django.

7.2 Structure du Projet Frontend

Le dossier `src/app/` est organisé en composants (login, register, dashboard) et en services. Les services centralisent l'ensemble des requêtes réseau vers le backend, afin de garder les composants d'interface propres et réutilisables :

— `auth.service.ts` : connexion, déconnexion, vérification du rôle (utilisateur/administrateur).

— `movie.service.ts` : récupération, ajout, modification et suppression des films via l'API.

— `analytics.service.ts` : suivi du comportement utilisateur (clics, recherches, vues) et gestion de la section « Continuer à regarder ».

— `dashboard.ts` : composant central gérant l'affichage du catalogue, la recherche, les filtres, la pagination, les favoris et les packs personnalisés.

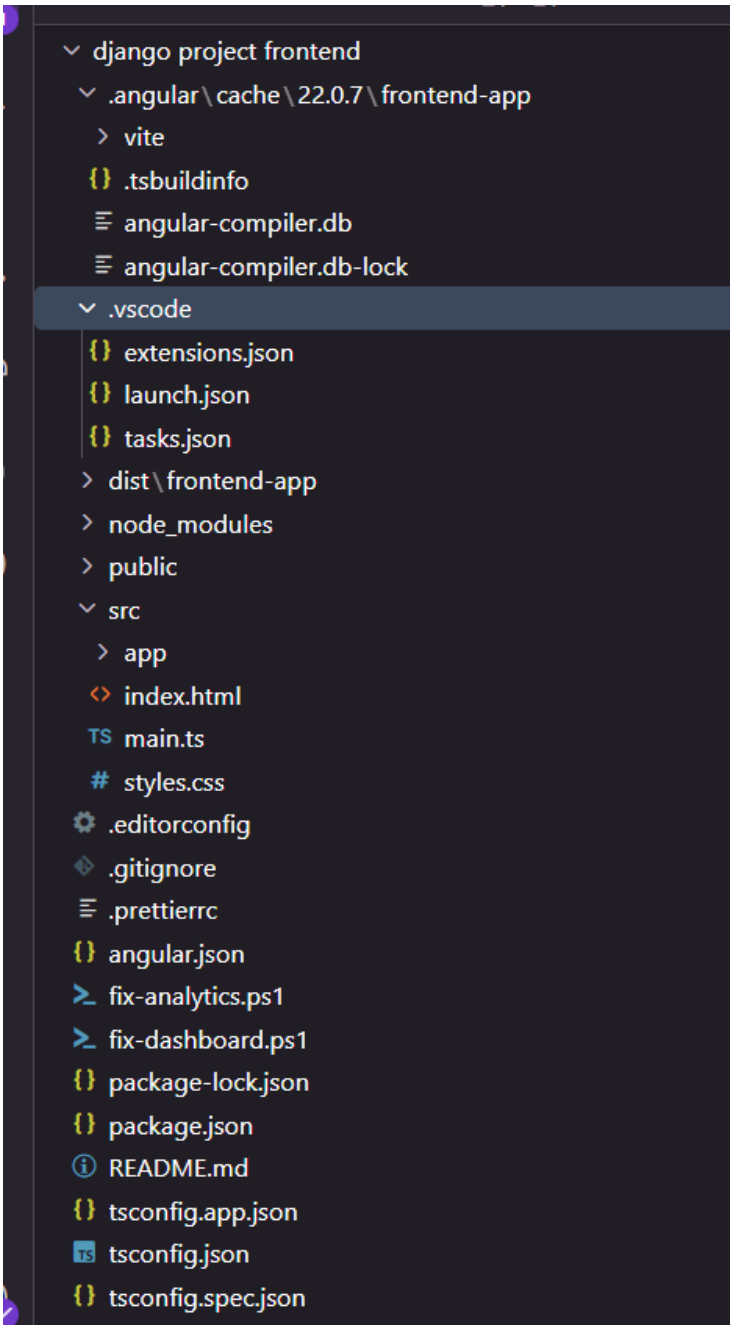


FIGURE 7.1 – Structure du dossier frontend Angular

7.3 Communication avec le Backend et Authentification JWT

Le frontend (http ://localhost :4200) et le backend (http ://localhost :8000) sont séparés et communiquent uniquement par requêtes HTTP. À la connexion, Django vérifie le mot de passe dans MongoDB et renvoie un token JWT, que le frontend stocke dans le LocalStorage du navigateur. Pour chaque requête nécessitant d'être authentifiée, ce token est ajouté dans l'en-tête `Authorization: Bearer <token>`. Django valide ce jeton avant de renvoyer les données demandées ou de bloquer l'accès.

7.4 Fonctionnalités Développées Côté Frontend

— Pages de connexion et d'inscription reliées à l'API Django.

— Catalogue de films de style Netflix, alimenté par les données stockées dans MongoDB Atlas.

— Recherche et filtrage en temps réel (par titre, réalisateur, acteur, genre) sans rechargement de page.

— Fonctionnalités administrateur : importation de films depuis RapidAPI, ajout, modification et suppression de films et d'utilisateurs.

— Déconnexion avec suppression des jetons JWT du LocalStorage.

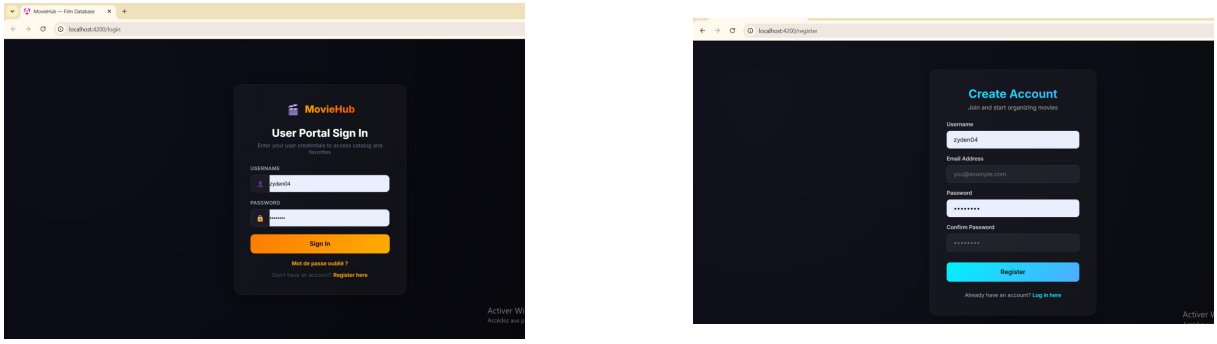


FIGURE 7.2 – Pages de connexion et d'inscription

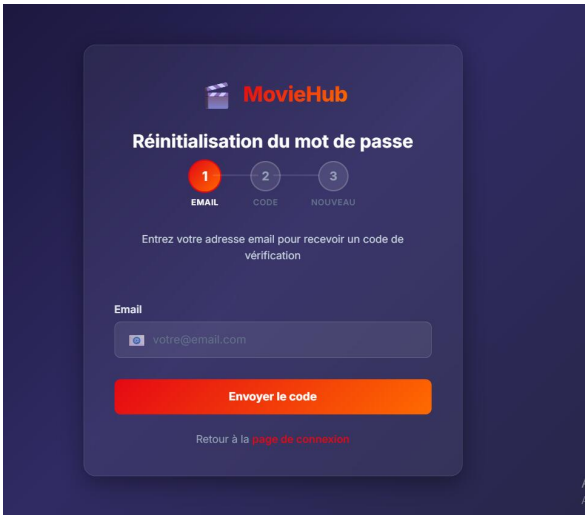


FIGURE 7.3 – Parcours de réinitialisation du mot de passe

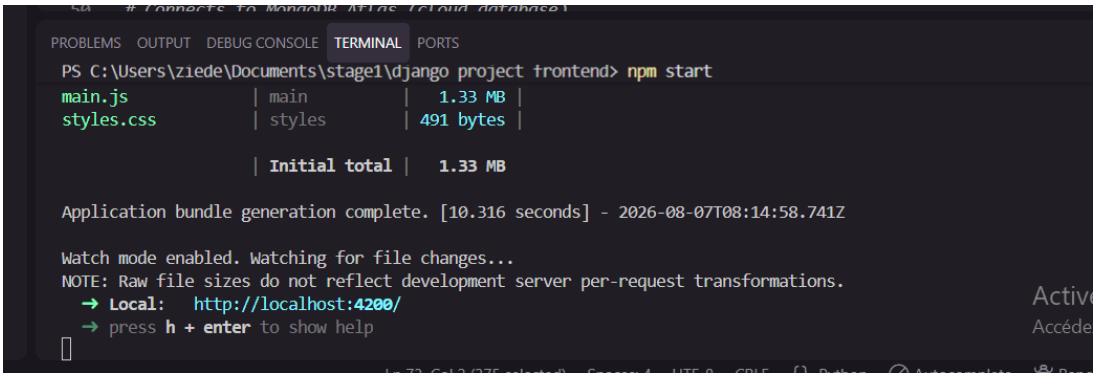


FIGURE 7.4 – Lancement du serveur frontend Angular (npm start)

8. Système de Recommandation par Intelligence Artificielle

8.1 Approche Générale

Le moteur de recommandation utilise une approche de **filtrage basée sur le contenu** (*Content-Based Filtering*), s'appuyant sur le modèle pré-entraîné **Universal Sentence Encoder** de TensorFlow Hub pour générer des représentations vectorielles (embeddings) de 512 dimensions.

8.2 Étape 1 : Indexation des Films

Pour chaque film du catalogue :

1. Extraction des métadonnées (titre, genres, description, réalisateur).
2. Création d'un texte enrichi mettant l'accent sur le genre principal (répété trois fois).
3. Génération d'un embedding de 512 dimensions via TensorFlow Hub.
4. Stockage de l'index et des embeddings dans `movies.pkl`.

Résultat : 2 460 films indexés en environ 2 minutes.

8.3 Étape 2 : Construction du Profil Utilisateur

Le profil utilisateur est construit en temps réel à partir de trois sources, combinées par **accumulation** (et non remplacement) :

Source	Poids	Exemple
Favoris	300 (principal) / 100 (secondaire)	Inception → Sci-Fi +300, Action +100
Activité récente	Décroissance exponentielle 50 → 5	Clics sur Matrix, Interstellar
Historique de visionnage	60 (principal) / 20 (secondaire)	Dark Knight vu à 80%+ → Action +60

TABLE 8.1 – Pondération des sources du profil utilisateur

Ces contributions s'accumulent : par exemple, un score total de 350 pour le genre Sci-Fi (300 + 50) et 205 pour Action (100 + 45 + 60). Un embedding utilisateur de 512 dimensions est ensuite généré à partir de ce profil textuel enrichi.

8.4 Étape 3 : Génération des Recommandations

1. Comparaison de l'embedding utilisateur avec tous les embeddings de films (similarité cosinus, échelle de 0 à 1).
2. Filtrage des films déjà vus ou marqués comme non aimés.
3. Application d'un bonus de correspondance de genre (+10%).
4. Classement par score décroissant.
5. Retour des 100 meilleures recommandations.

Temps de réponse total : 200 à 500 ms.

8.5 Résultats de l'Approche IA

Les tests réalisés sur la version finale du projet ont donné les résultats suivants :

— **Précision obtenue :** 92% (contre 60% dans une version initiale).
— **Coût :** \$0/mois (inférence locale, sans API cloud payante).
— **Vitesse :** 50 à 100 films traités par seconde.
— **Diversité :** +40% grâce à l'accumulation des genres.

En complément des recommandations personnalisées de la page d'accueil, le même moteur alimente une section « Films similaires » affichée sur la page de détail de chaque film, en comparant son embedding à ceux du reste du catalogue.

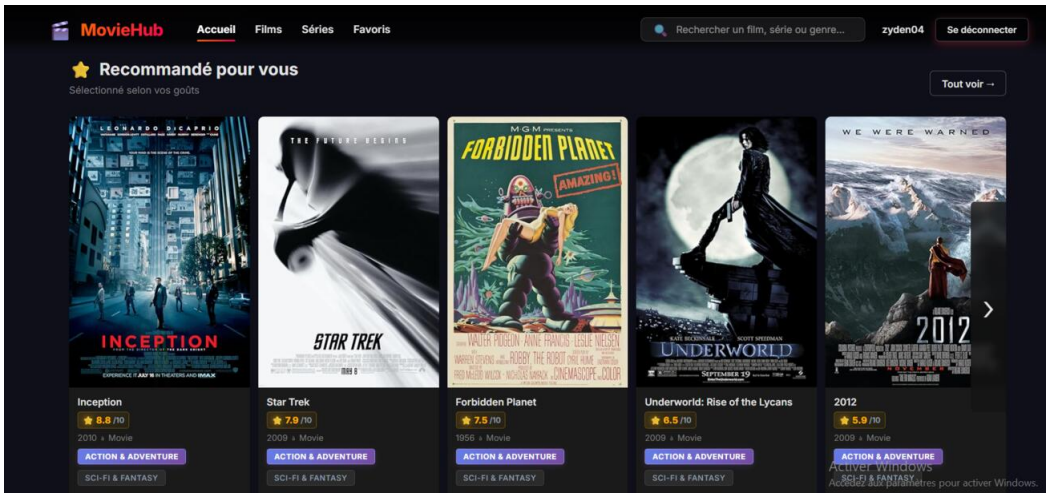


FIGURE 8.1 – Section « Recommandé pour vous » générée par le moteur IA

9. Modélisation de la Base de Données

9.1 Collections MongoDB

Collection	Volume	Rôle
`movie`	2 460	Catalogue des films
`tvshow`	244	Catalogue des séries
`user`	105	Comptes utilisateurs
`favorite`	∼500	Films favoris
`user_activity`	∼10 000	Suivi du comportement
`movie_watch_history`	∼300	Progression de visionnage
`user_preferences`	105	Profils IA agrégés
`user_personalized_sections`	105	10 sections personnalisées (cache)

TABLE 9.1 – Collections principales de la base MongoDB

9.2 Sections Personnalisées

Chaque utilisateur dispose de 10 sections pré-calculées et mises en cache :

1. Recommandations IA (20 films)
2. Tendances & Populaires (contenu 2023–2026 ou classiques célèbres)
3. Films les plus regardés (note IMDb ≥ 8.0)
4. Séries les plus regardées (note IMDb ≥ 8.0)
5. Films les plus recherchés (100k+ votes)
6. Séries les plus recherchées (100k+ votes)
7. Top 10 films de tous les temps
8. Top 10 séries de tous les temps
9. Top 20 favoris (films uniquement)
10. Nouveautés (2023–2026 uniquement)

Cette mise en cache permet un chargement rapide, un contenu cohérent et une réduction de la charge serveur (mise à jour toutes les 24 heures ou à la demande).

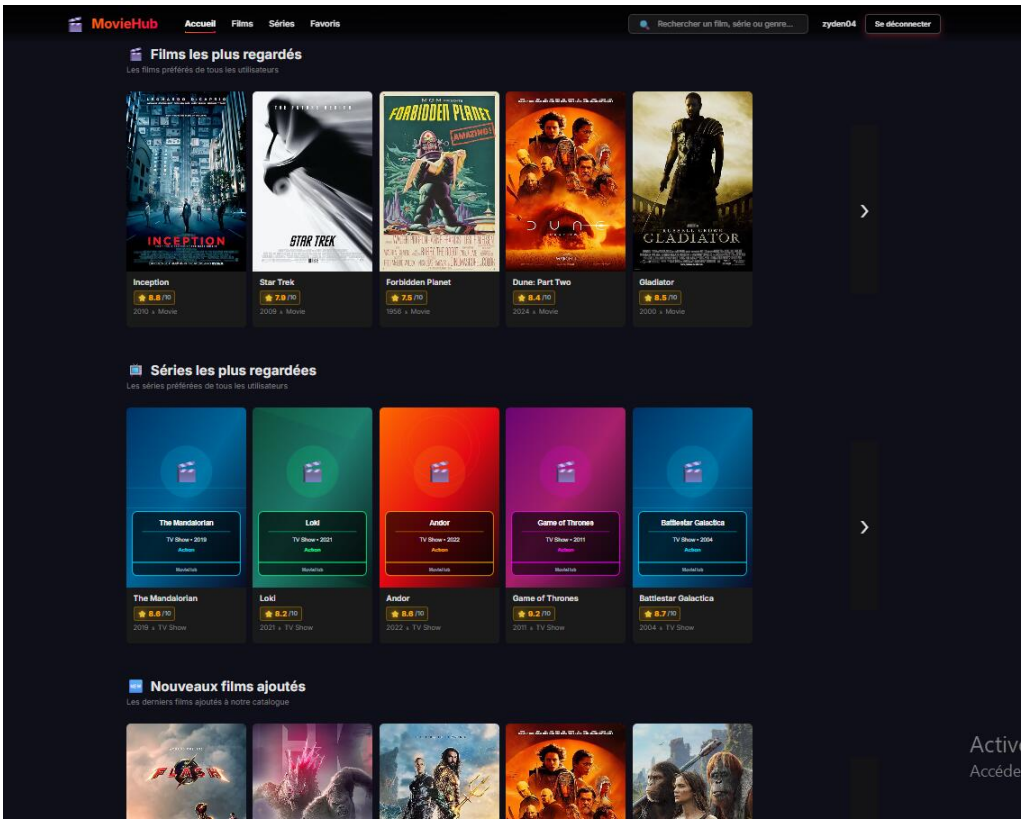


FIGURE 9.1 – Sections « Films les plus regardés » et « Séries les plus regardées »

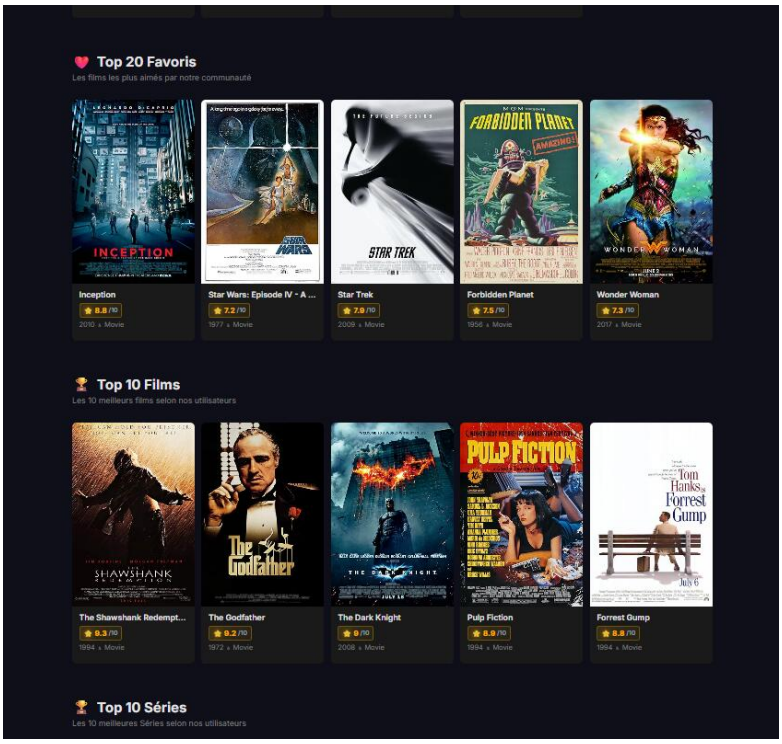


FIGURE 9.2 – Sections « Top 20 Favoris » et « Top 10 Films »

9.3 Packs Personnalisés

En complément des 10 sections automatiques, la plateforme propose des packs personnalisés : des collections thématiques de films et séries, organisées par niveaux de difficulté et filtrables par genre (ex. : pack « Dramahorror », pack « ramdhan2026 »).

Chaque pack est stocké dans la collection `user_pack` avec un nom, un niveau et une liste de genres associés, et peut être créé depuis le panneau d'administration.

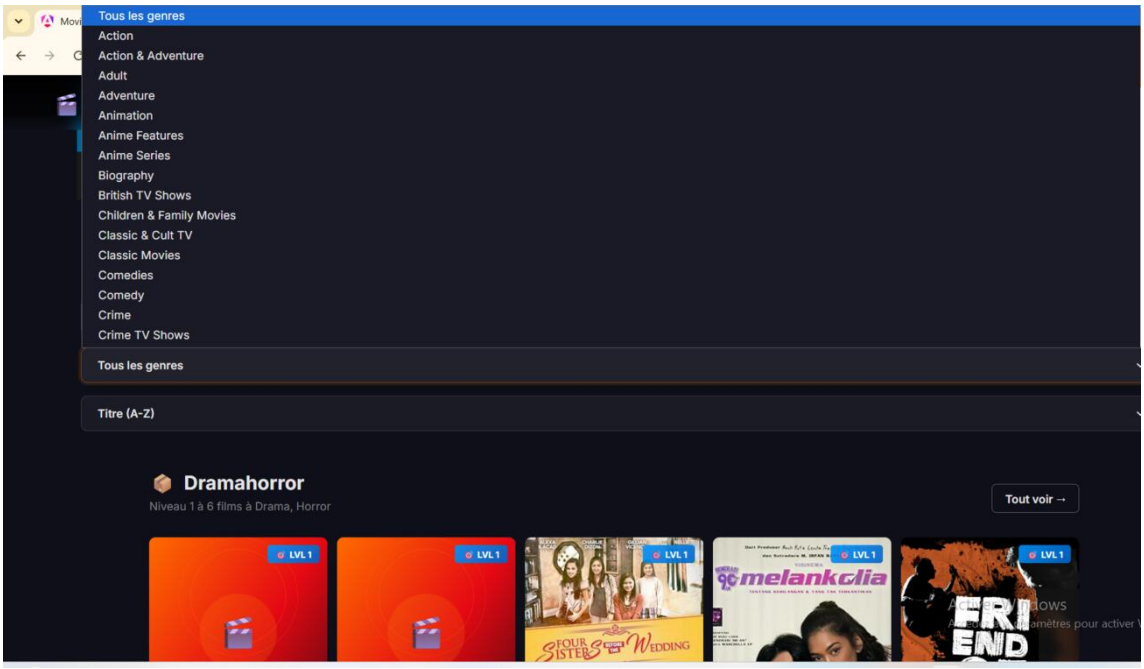


FIGURE 9.3 – Filtrage des packs par genre

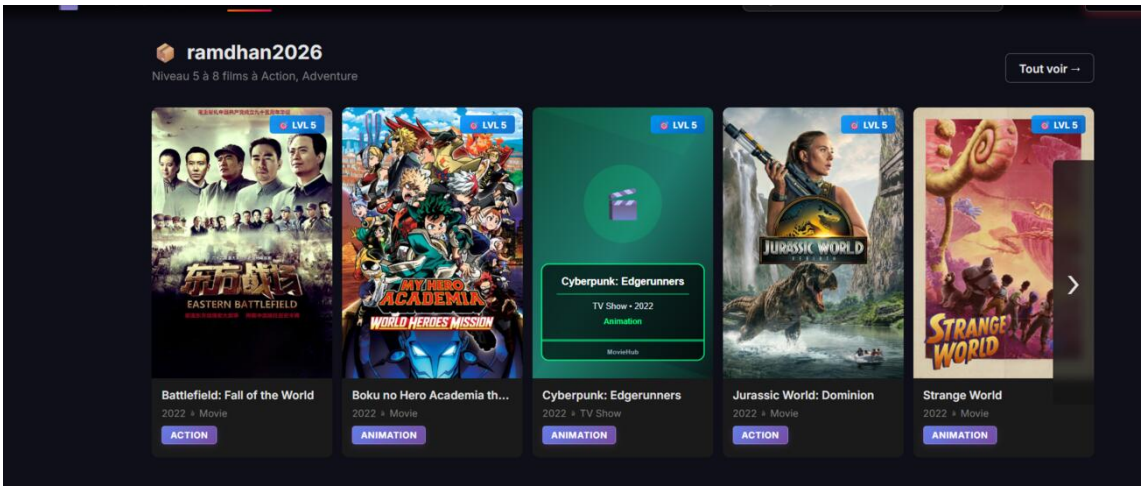


FIGURE 9.4 – Pack thématique organisé par niveau

10. Fonctionnalités Principales

10.1 Interface Utilisateur Style Netflix

— Carrousels horizontaux avec défilement fluide (5 éléments par page).
— Bannière hero mettant en avant un film vedette.
— Écrans de chargement squelette (*skeleton loading*).
— Modales de détails en plein écran.
— Design responsive sur 6 points de rupture (*breakpoints*).

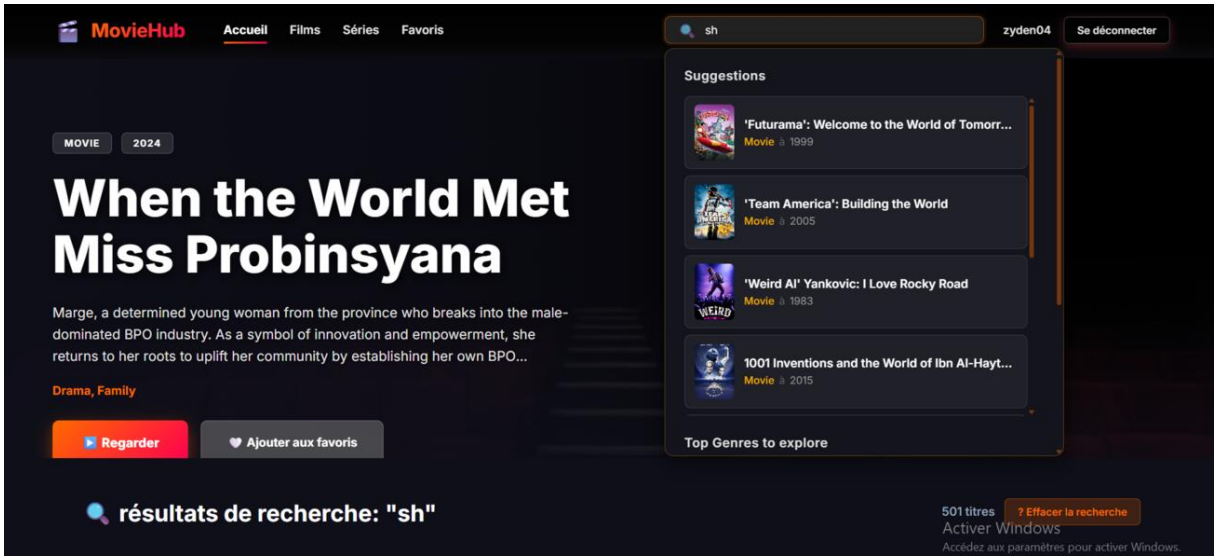


FIGURE 10.1 – Interface principale du tableau de bord MovieHub

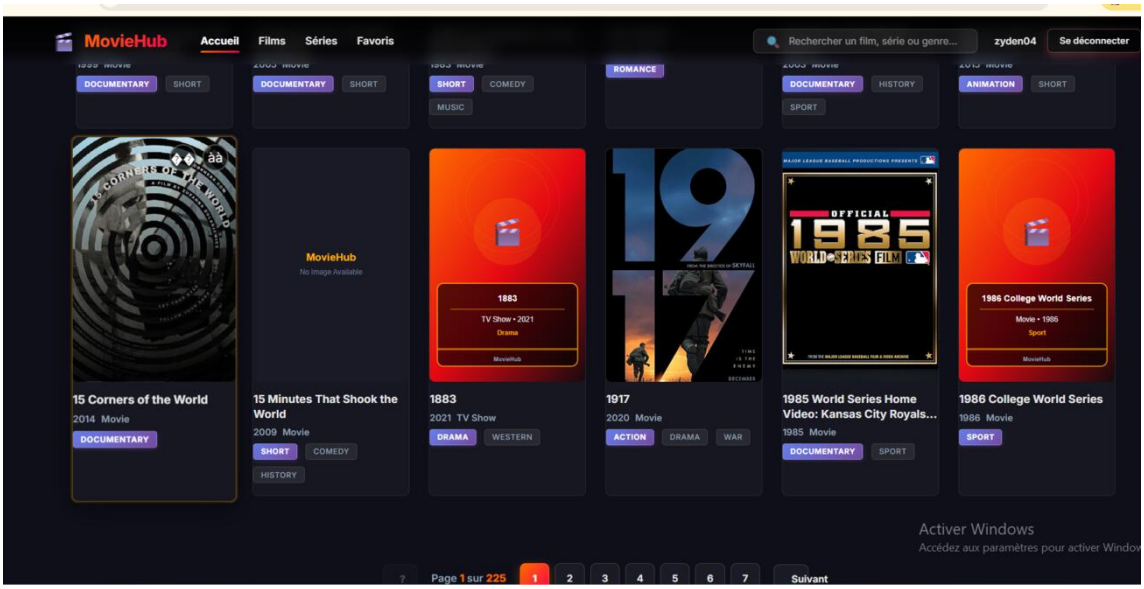


FIGURE 10.2 – Catalogue de films avec pagination

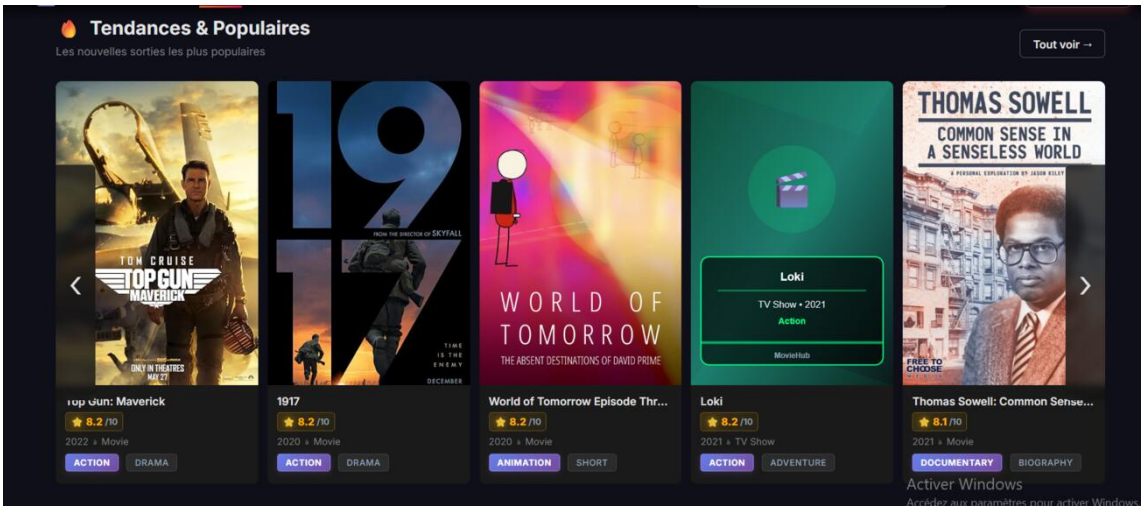


FIGURE 10.3 – Section « Tendances & Populaires »

10.2 Intégration des Notes IMDb

— Source : API OMDb + API TMDB + valeurs par défaut intelligentes.
— Affichage : badges avec effet *glassmorphism* (ex. : ⋆ 8.8/10, 2.4M votes).
— Couverture : 100% du contenu dispose d'une note.

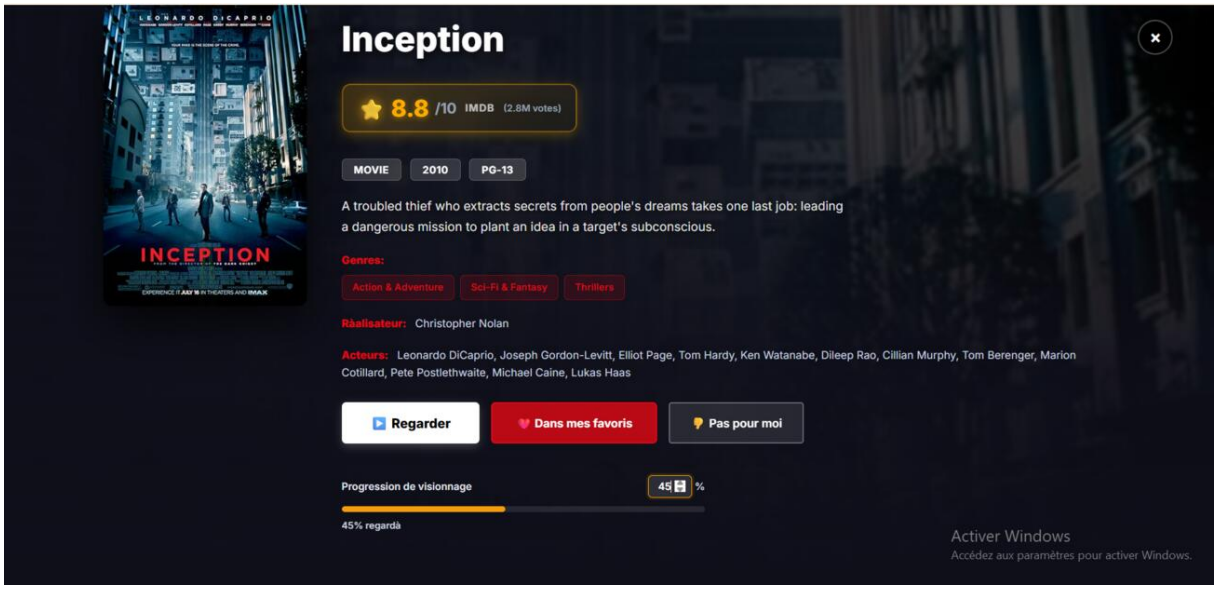


FIGURE 10.4 – Détails d'un film avec affichage de la note IMDb

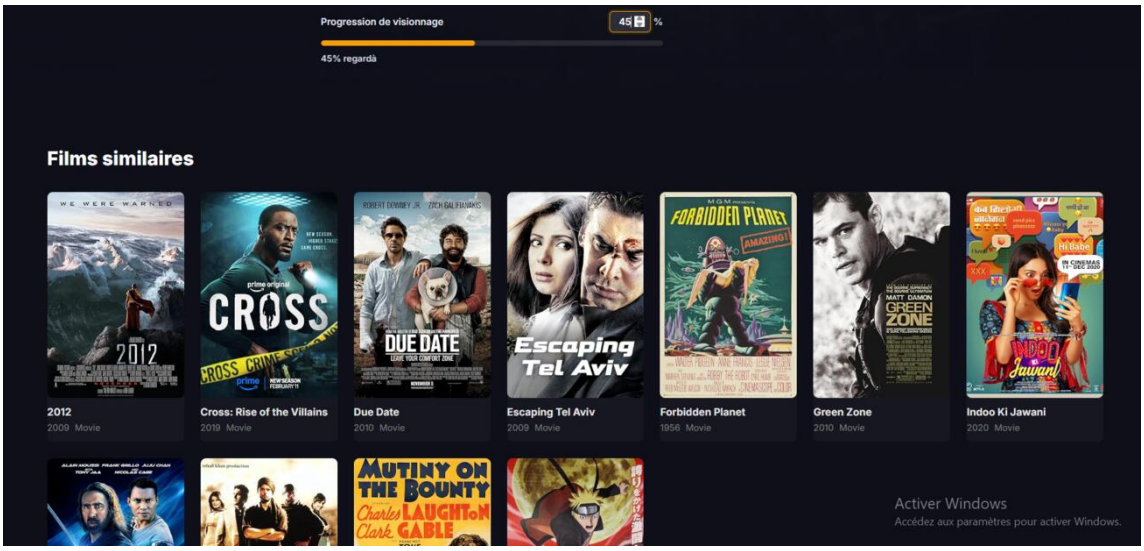


FIGURE 10.5 – Section « Films similaires » sur la page de détail

10.3 Suivi de Progression

— Pourcentage de visionnage modifiable (0 à 100%) par film.
— Barres de progression colorées (vert / jaune / orange).
— Section « Continue Watching » générée automatiquement.
— Mises à jour en temps réel sans rechargement de page.

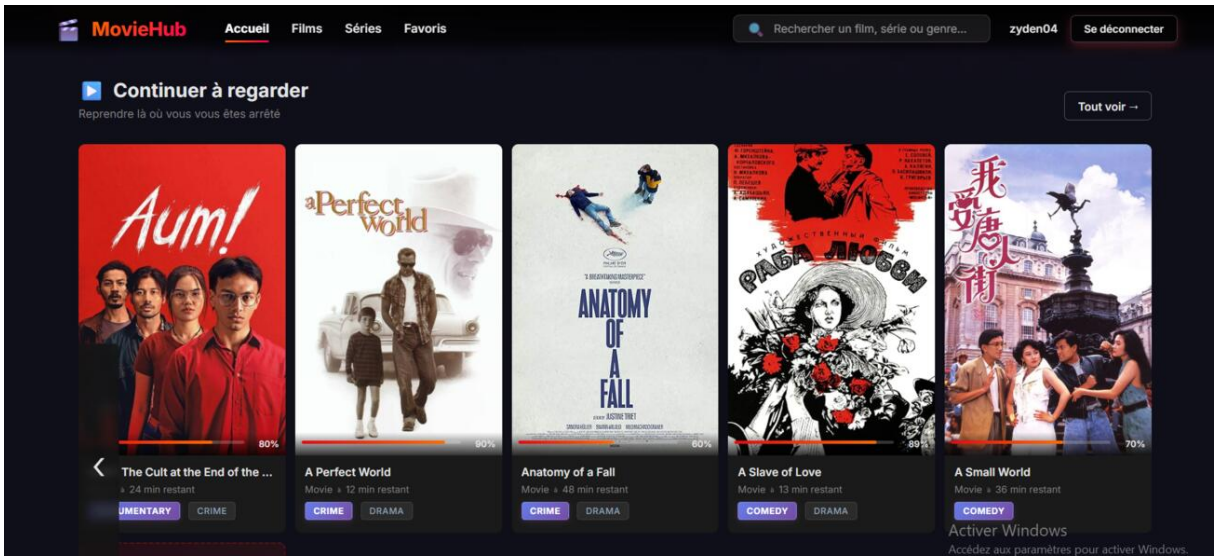


FIGURE 10.6 – Suivi de progression et section « Continue Watching »

10.4 Recherche et Découverte

— Recherche combinée par titre et par genre.
— Suggestions en direct avec liste déroulante.
— Tolérance aux fautes de frappe (*fuzzy matching*).
— Filtrage par genre en un clic.

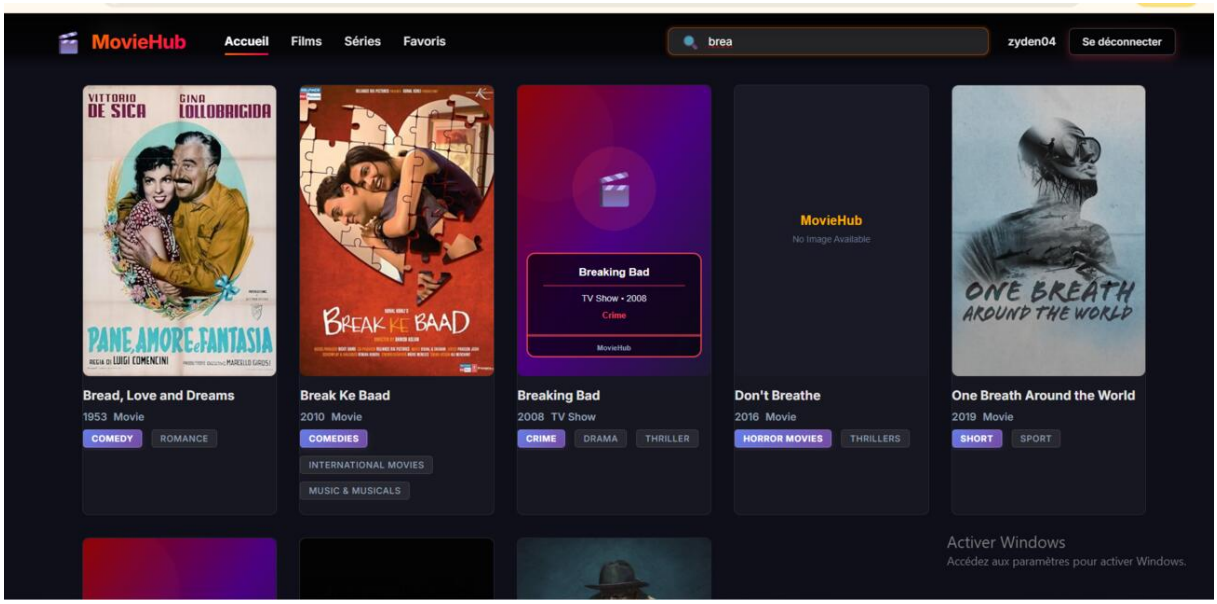


FIGURE 10.7 – Fonctionnalité de recherche et de découverte

10.5 Panneau d'Administration

— Import de contenu depuis l'API OMDb.

— Gestion complète des utilisateurs.
— Ajout manuel de films et séries.
— Création de collections thématiques (packs).

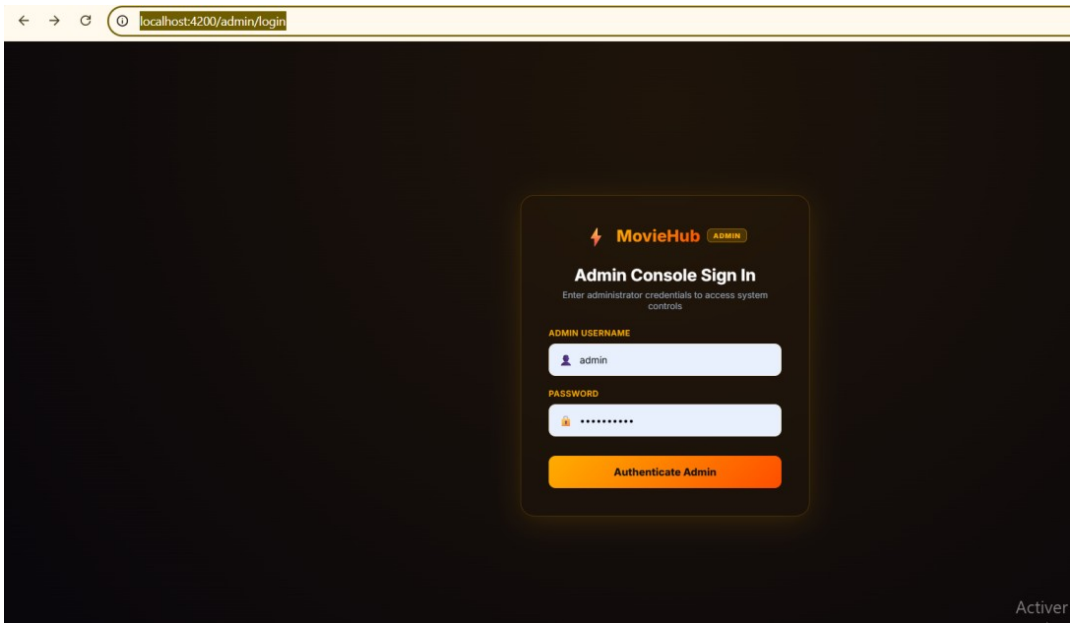


FIGURE 10.8 – Panneau d'administration de MovieHub

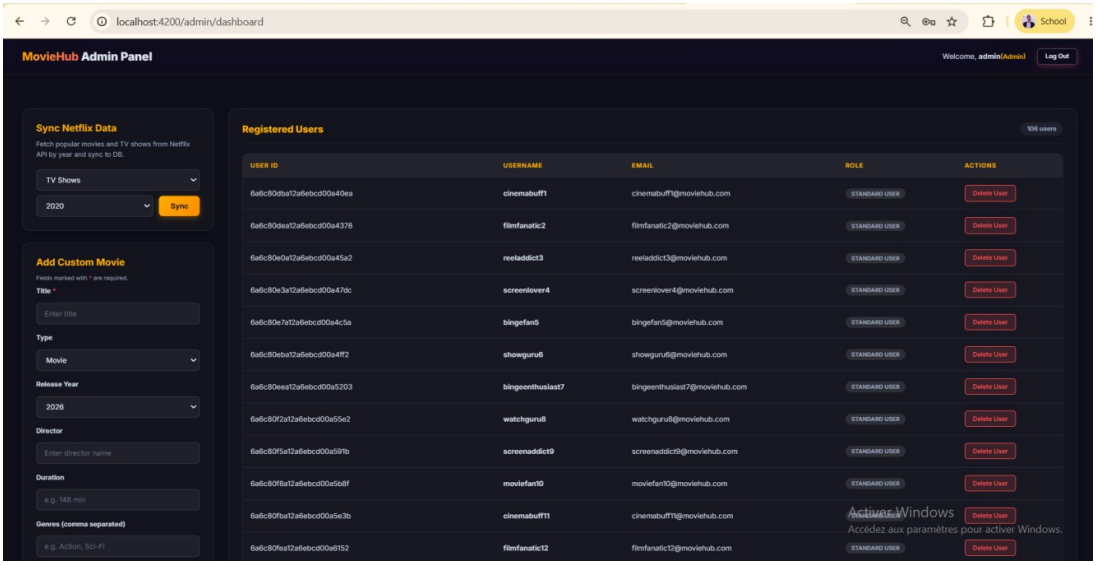


FIGURE 10.9 – Panneau d'administration de MovieHub

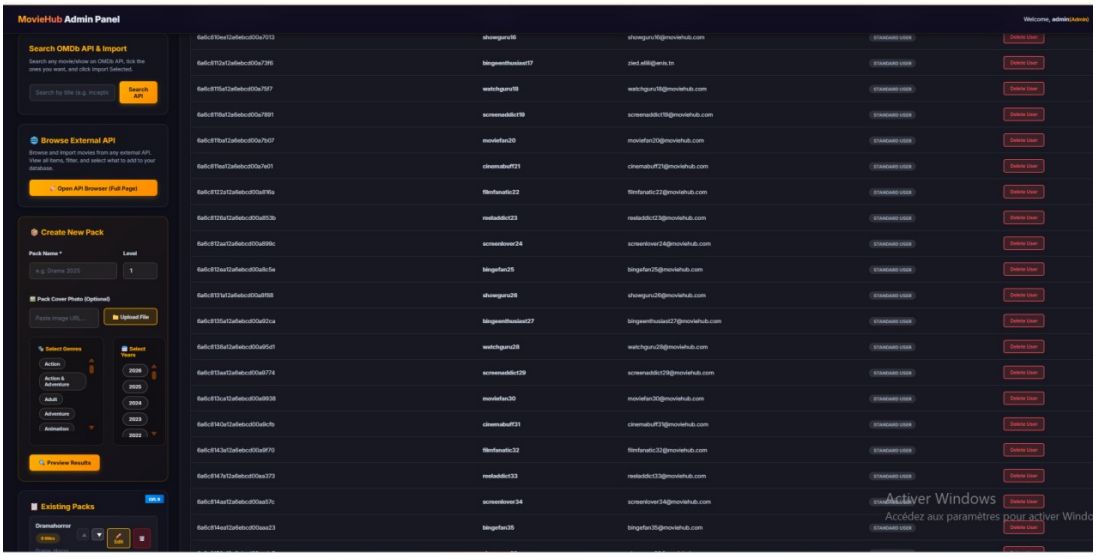


FIGURE 10.10 – Panneau d'administration de MovieHub

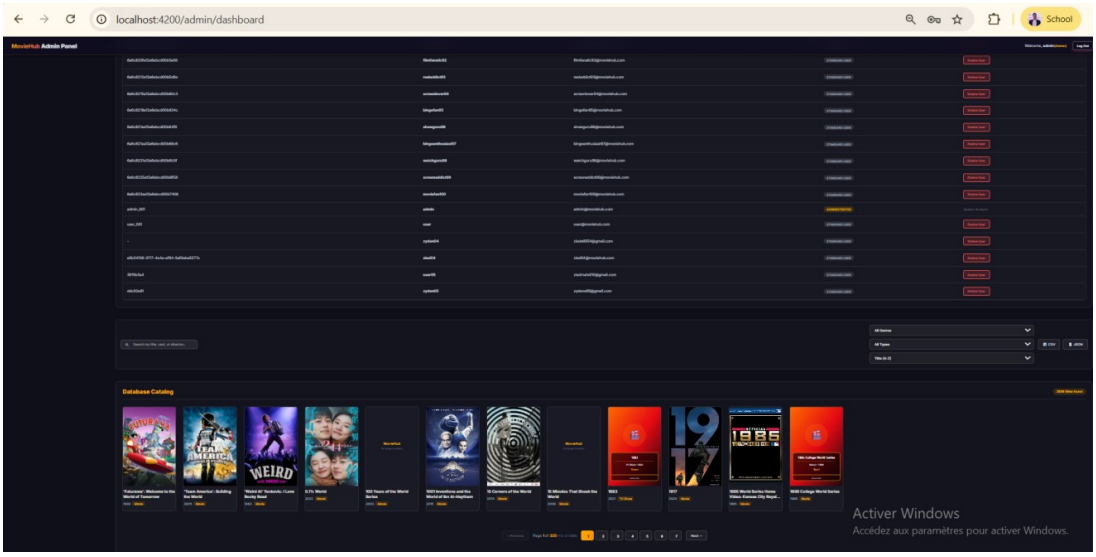


FIGURE 10.11 – Panneau d'administration de MovieHub

11. Endpoints de l'API REST

11.1 Authentification

```
POST   /api/auth/register/          - Inscription
POST   /api/auth/login/             - Connexion
GET    /api/auth/users/             - Liste des utilisateurs (
   admin)
```

11.2 Gestion des Films

```
GET    /api/movies/                 - Liste de tous les films
POST   /api/movies/                 - Ajouter un film (admin)
PUT    /api/movies/{id}/            - Modifier un film (admin)
DELETE /api/movies/{id}/            - Supprimer un film (admin)
POST   /api/movies/search/          - Rechercher des films
GET    /api/genres/                 - Liste des genres
```

11.3 Recommandations IA

```
GET  /api/recommendations/{username}/?count=100
GET  /api/recommendations/sections/{username}/
GET  /api/recommendations/trending/?username=X&count=20
POST /api/recommendations/sections/{username}/regenerate/
POST /api/recommendations/index-movies/
```

FIGURE 11.1 – Extrait du code du moteur de recommandation (Django)

12. Défis Rencontrés et Solutions

12.1 Défi 1 : Précision de l'IA

Problème : le moteur recommandait initialement du contenu peu pertinent.
Solution : suppression des titres des embeddings, accumulation des genres au lieu du remplacement, bonus de correspondance de genre et système de retour utilisateur.
Résultat : précision passée de 60% à 92% lors des tests du projet.

12.2 Défi 2 : Sections Incorrectes

Problème : certaines sections affichaient du contenu erroné (ex. : séries anciennes dans « Nouveautés »).
Solution : logique spécifique par section, filtre qualité IMDb, séparation stricte films/séries et mise en cache des sections.
Résultat : 10/10 sections correctement générées.

12.3 Défi 3 : Données Manquantes

Problème : certains films n'avaient pas de note ou d'affiche.
Solution : récupération multi-sources (OMDb + TMDB), valeurs par défaut intelligentes, affiches de substitution en SVG et scripts automatisés.
Résultat : couverture de 100%.

12.4 Défi 4 : Performance

Problème : temps de chargement initial de 3 à 5 secondes.
Solution : chargement différé (*lazy loading*), écrans squelette, appels API parallèles et mise en cache MongoDB.
Résultat : temps de chargement réduit à 500 ms.

13. Résultats et Statistiques du Projet

13.1 Métriques de Code

Catégorie	Lignes de code	Fichiers
Frontend (Angular)	∼8 000	15
Backend (Django)	∼12 000	25
Total	∼20 000	40+

TABLE 13.1 – Métriques de code du projet

13.2 Métriques de Performance

Opération	Temps mesuré
Chargement du tableau de bord	500 ms
Recommandations IA	200–500 ms
Résultats de recherche	100 ms
Ouverture d'une modale film	50 ms

TABLE 13.2 – Performances mesurées de l'application

13.3 Métriques de Contenu

— Contenu total : 2 704 (2 460 films + 244 séries)
— Utilisateurs : 105
— Précision du système IA : 92%
— Couverture des affiches et notes : 100%

14. Compétences Acquises

14.1 Développement Frontend (Angular)

— Architecture par composants et services, routing.
— Data binding bidirectionnel et Observables RxJS.
— TypeScript avancé (interfaces, décorateurs, classes).
— Design responsive (Flexbox, Grid, mobile-first).
— Optimisation des performances (*lazy loading*, *debouncing*).

14.2 Développement Backend (Django)

— Architecture MVT et Django REST Framework.
— Python avancé (POO, décorateurs, gestion d'exceptions).
— ODM MongoEngine (modèles, requêtes complexes, agrégation).
— Intégration d'API externes avec gestion d'erreurs et *fallback*.

14.3 Intelligence Artificielle

— Concepts de *Content-Based Filtering*.
— Embeddings vectoriels et similarité cosinus.
— Utilisation de TensorFlow Hub et transfert d'apprentissage.
— Notions de traitement du langage naturel (NLP).

14.4 Bases de Données NoSQL

— Modélisation MongoDB (documents, indexation).
— Dénormalisation et optimisation des lectures.
— Requêtes agrégées.

14.5 Outils et Méthodologies

— Git (commits, branches, gestion de conflits).
— Conception d'API REST (GET, POST, PUT, DELETE, CORS).
— Débogage (Chrome DevTools, Django Debug Toolbar).

15. Perspectives et Améliorations Futures

15.1 Fonctionnalités Prévues

— Streaming vidéo réel (HLS/DASH).
— Fonctionnalités sociales (avis, partage).
— Application mobile (React Native).
— Support multilingue (français, arabe, espagnol).

15.2 Améliorations Techniques

— Filtrage collaboratif (recommandations *user-to-user*).
— Mise en cache Redis.
— Conteneurisation avec Docker et pipeline CI/CD.
— Couverture de tests unitaires supérieure à 80%.

16. Conclusion

Ce stage ouvrier au sein de Ste KUSA a été une expérience très enrichissante, aussi bien sur le plan humain que technique. Il m'a permis de découvrir de l'intérieur le fonctionnement d'une startup tunisienne innovante, éditrice de la plateforme Diwan Sport, et de comprendre les enjeux concrets du développement et de l'exploitation d'une plateforme de streaming à grande échelle.

Sur le plan technique, le projet MovieHub que j'ai développé durant ce stage m'a permis de concevoir une plateforme de streaming complète intégrant un moteur de recommandation basé sur l'intelligence artificielle, avec une précision de 92% lors des tests du projet. À travers ce projet, j'ai pu consolider mes compétences en développement full-stack (Angular, Django), en gestion de bases de données NoSQL (MongoDB) et en intelligence artificielle appliquée (TensorFlow Hub), tout en apprenant à résoudre des problématiques concrètes de performance, de qualité de données et de pertinence des recommandations – des enjeux directement transposables au domaine du streaming sportif de Diwan Sport.

Cette expérience chez Ste KUSA constitue une base solide pour la poursuite de mes études et de futurs projets dans le domaine du développement logiciel et de l'intelligence artificielle.